

Spécification technique — Orchestrateur de suivi des missions

Oct 6, 2026 · @Saouadogo Salifou

1. Contexte

Le suivi des missions de transport repose aujourd'hui sur des agents de suivi d'exploitation qui surveillent chaque camion à la main dans le TMS. L'orchestrateur automatise cette surveillance avec un agent IA, sans modifier le TMS.

Le métier. Une mission suit un cycle de vie fixe : déplacement vers le point de chargement, chargement, trajet, déchargement. L'agent de suivi compare en continu la position GPS du camion à l'itinéraire planifié. Il repère les dérives, les retards, les immobilisations et les incidents. Il prévient ensuite les bons interlocuteurs selon la gravité : chauffeur, agent de suivi, superviseur, manager, direction.

Le problème. Ce travail est répétitif, continu et difficile à tenir sur un grand nombre de missions simultanées. Une anomalie vue trop tard coûte un retard de livraison ou un client mécontent. Chaque TMS expose aussi ses données différemment, ce qui rend toute automatisation coûteuse à intégrer.

Le TMS. Le TMS est un système tiers que nous ne contrôlons pas. Quatre contraintes en découlent :

1. **Lecture seule** : l'orchestrateur ne modifie jamais le TMS. Son API n'autorise que des requêtes `GET`.
2. **Pull uniquement** : le TMS ne pousse rien (ni webhook, ni événement). L'orchestrateur l'interroge à son rythme, avec les filtres `updated_since` et `since`.
3. **Accès via le serveur MCP** : l'agent n'appelle jamais l'API du TMS directement.
4. **Données de l'agent chez l'agent** : alertes, notifications, journal et appels chauffeur sont stockés dans la base de l'orchestrateur, jamais dans le TMS.

État actuel. Les vraies règles métier seront formalisées lors d'entretiens avec les agents de suivi. En attendant, un POC tourne sur un TMS simulé (API FastAPI, simulateur de missions, interface web) pour valider l'architecture, la boucle agentique et le découplage entre l'agent, le TMS et les règles.

2. Objectifs et périmètre

L'orchestrateur doit surveiller en temps réel les missions de n'importe quel TMS connecté, détecter les anomalies à partir de règles métier explicites, notifier les bons niveaux de management et expliquer chaque alerte. Il vise à augmenter la productivité des agents de suivi, voire à absorber une partie de leur charge.

Objectifs de l'agent IA

1. **Surveillance continue** de toutes les missions actives, sans intervention humaine.
2. **Détection déterministe et auditable** : une alerte naît toujours d'une règle métier YAML versionnée, jamais d'une décision brute du LLM.
3. **Escalade selon la gravité** : chaque règle définit les rôles, les niveaux de management (M+0 à M+n) et les canaux à notifier.
4. **Enquête et explication** : le LLM analyse chaque nouvelle alerte et joint une évaluation structurée (cause probable, risque, actions recommandées, preuves).
5. **Assistant opérateur** : répondre en langage naturel aux questions sur les missions et les alertes.
6. **Indépendance vis-à-vis du TMS** : brancher un nouveau TMS revient à écrire un connecteur. L'agent découvre les tools dynamiquement et n'a aucun code spécifique au TMS.
7. **Règles aux mains du métier** : lisibles, modifiables et testables par l'équipe métier, rechargées à chaud.
8. **Confidentialité** : LLM local par défaut, aucune donnée ne sort de l'infrastructure sans autorisation explicite.

Périmètre du POC

- Serveur MCP (accès TMS en lecture seule, base de l'agent, notifications simulées).
- Agent IA en ligne de commande : boucle de surveillance, enquêtes, assistant, rapport.
- Moteur de règles déclaratif avec les 6 règles de démonstration et leurs scénarios de test.
- TMS simulé avec 4 scénarios : nominal, dérive, immobilisation, incident critique.

Hors périmètre du POC

- Envoi réel de SMS, e-mails ou appels (l'envoi est simulé et enregistré en base).
- Écriture dans le TMS (contrainte permanente, pas seulement du POC).
- Interface opérateur web, API HTTP de l'agent, authentification des utilisateurs.

- Vraies règles métier, attendues après les entretiens avec les agents de suivi.

Critères de succès du POC

Critère	Cible
Anomalies des scénarios simulés détectées	100 % (incident, dérive, retard ETA, immobilisation)
Alertes en double pour une même situation	0 (déduplication et cooldown)
Fausse clôture d'alerte	0 : clôture seulement sur instantané complet
Scénarios métier des règles	20/20 réussis
Code spécifique au TMS dans l'agent	Aucun
Délai de détection	1 cycle de surveillance (10 s par défaut)
Analyse jointe à chaque nouvelle alerte	Oui, en arrière-plan (20 à 70 s avec Qwen3.5-2B)

3. Acteurs et cas d'usage

Sept acteurs interagissent avec l'orchestrateur : cinq destinataires d'alertes, alignés sur les niveaux de management, plus l'équipe métier et l'équipe technique.

Acteur	Rôle TMS	Niveau	Canal typique	Ce qu'il attend du système
Chauffeur	driver	M+0	Appel	Être contacté quand son camion dérive ou s'arrête anormalement
Agent de suivi	ops_agent	M+1	In-app	Recevoir chaque alerte avec son analyse, interroger l'assistant, clore les alertes
Superviseur	ops_supervisor	M+2	SMS	Être prévenu des anomalies graves (dérive, immobilisation, incident)

Acteur	Rôle TMS	Niveau	Canal typique	Ce qu'il attend du système
Manager d'exploitation	ops_manager	M+3	Appel	Être prévenu des incidents critiques et des retards majeurs
Direction	director	M+4	E-mail	Être informée des incidents critiques de sécurité
Équipe métier	—	—	Fichiers YAML	Écrire, tester et faire évoluer les règles sans développeur
Équipe technique	—	—	CLI, logs	Brancher un TMS, déployer, exploiter, superviser

Les destinataires ne sont pas codés en dur : le serveur MCP les résout dans le TMS à chaque notification. Le rôle `driver` donne le chauffeur de la mission ; les autres rôles donnent les utilisateurs qui portent ce rôle.

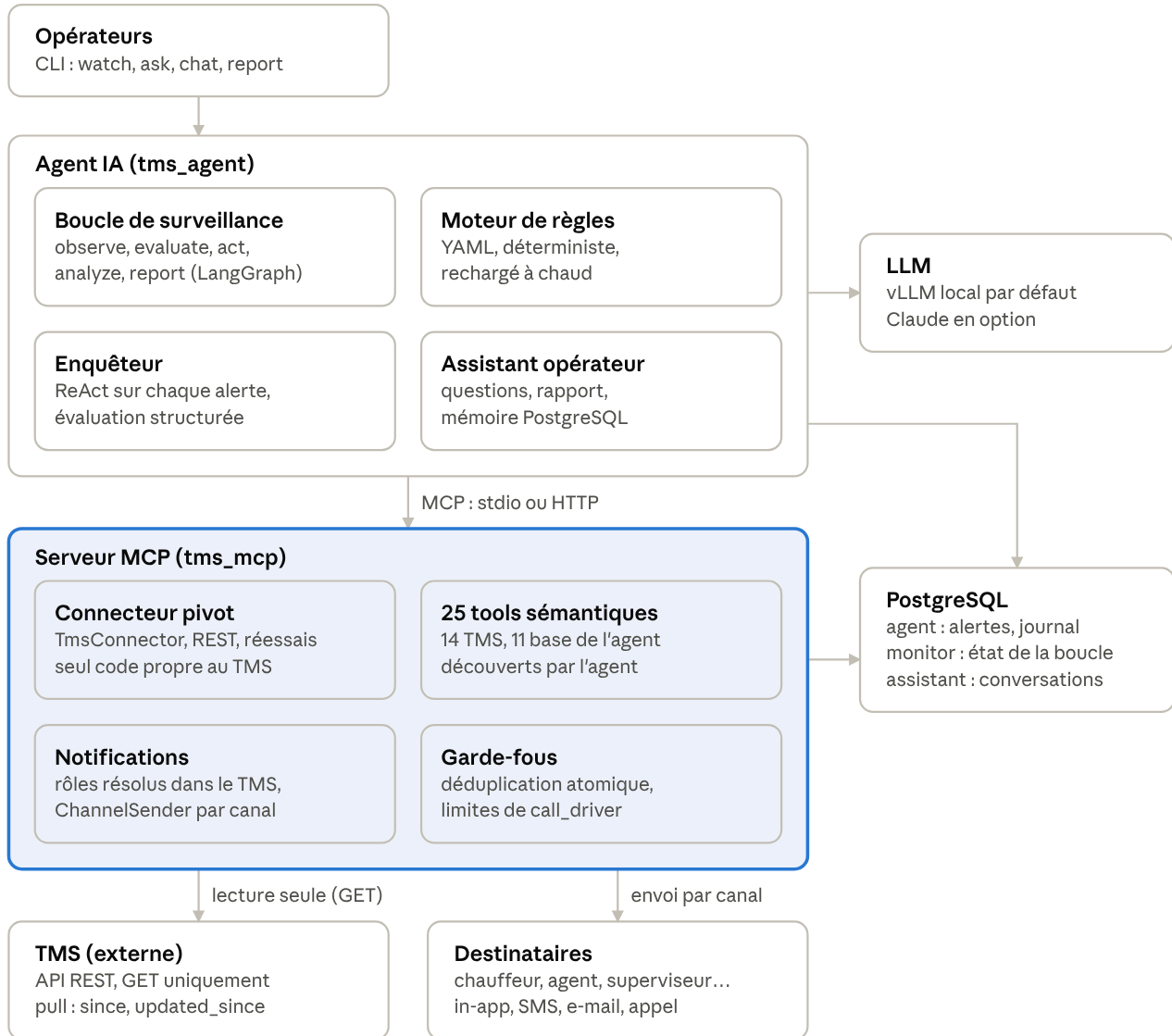
Cas d'usage principaux

1. **Détecter et notifier** : la boucle détecte une dérive de plus de 5 km depuis plus de 20 min. Elle crée l'alerte, appelle le chauffeur, notifie l'agent de suivi en in-app et le superviseur par SMS.
2. **Expliquer une alerte** : le LLM enquête en arrière-plan (instantané de mission, événements, positions, évaluation des règles) et joint son évaluation à l'alerte.
3. **Clore automatiquement** : quand la condition disparaît (le camion revient sur l'itinéraire) ou que la mission se termine, l'alerte d'état est close.
4. **Interroger l'assistant** : « Quelles missions sont à risque ? », « Pourquoi la mission MSN-2 est-elle en alerte ? ». L'assistant répond en s'appuyant sur les tools MCP et garde le fil de la conversation.
5. **Produire un rapport de situation** : synthèse des missions actives, des alertes ouvertes et des actions menées.
6. **Faire évoluer une règle** : l'équipe métier modifie un seuil, rejoue les scénarios, puis dépose le fichier. La règle est active au cycle suivant, sans redémarrage.
7. **Brancher un nouveau TMS** : l'équipe technique écrit un connecteur vers le contrat pivot. Ni l'agent ni les règles ne changent.

4. Architecture d'ensemble

L'orchestrateur se compose de deux services Python, l'agent IA et le serveur MCP, autour d'une base PostgreSQL. Le serveur MCP est la seule frontière avec le TMS : c'est ce qui rend l'agent indépendant du TMS.

L'agent ne voit le TMS qu'à travers le serveur MCP



architecture · agent, serveur MCP, TMS, LLM, PostgreSQL

À chaque cycle, l'agent lit le TMS via les tools MCP, évalue les règles, puis demande au serveur MCP de créer l'alerte et de notifier. Le LLM n'intervient qu'après, pour enquêter ou répondre à un opérateur. Le serveur MCP possède le schéma `agent` ; l'agent possède les schémas `monitor` et `assistant`.

5. Spécification fonctionnelle

Les règles décident, le LLM explique. La détection et la notification ne dépendent jamais du LLM : sans lui (panne, quota, pas de clé), l'agent continue d'alerter selon les règles, seule l'analyse est sautée.

5.1 Surveillance continue

- **F1.** L'agent exécute un cycle toutes les `poll_interval_s` secondes (10 s par défaut).
- **F2.** Chaque cycle lit les missions en cours, puis l'instantané complet de chacune (mission, événements, véhicule, écart, ETA, chauffeur) en parallèle, avec au plus `mission_concurrency` appels simultanés (6 par défaut).
- **F3.** L'heure de référence est l'heure du TMS (`tms_time`), jamais l'horloge locale. Le système supporte ainsi le temps accéléré du simulateur et la remise à zéro de son horloge.
- **F4.** Une mission illisible ou un instantané partiel est signalé dans le bilan du cycle, sans arrêter le cycle.
- **F5.** Si le TMS ou le serveur MCP est injoignable, le cycle est signalé en échec, l'agent attend avec un backoff exponentiel et se reconnecte automatiquement.

5.2 Évaluation des règles

- **F6.** Les règles **d'état** sont évaluées à chaque cycle sur l'instantané de chaque mission (ex. dérive, retard ETA, immobilisation).
- **F7.** Les règles **d'événement** sont évaluées une seule fois par événement nouveau (ex. message chauffeur critique). Un événement n'est marqué traité qu'une fois ses alertes enregistrées ; un échec est repris au cycle suivant.
- **F8.** Les règles sont rechargées à chaud quand un fichier change. Une règle invalide est signalée et l'ancien jeu reste actif.
- **F9.** Les déclenchements sont traités par sévérité décroissante (`critical` → `low`).

5.3 Alertes

- **F10.** Chaque déclenchement `emit_alert` crée une alerte avec la règle et sa version, la sévérité, la catégorie, le titre et le message rendus, et les données observées (champs `observe` de la règle).

- **F11. Pas de doublon** : le serveur MCP applique atomiquement une clé de déduplication et un cooldown, en temps TMS. Une alerte supprimée est comptée dans le bilan avec sa raison.
- **F12.** Les actions `log_only` et `create_report` sont journalisées sans alerte ni notification.
- **F13. Clôture automatique** (`auto_resolve` , activée par défaut) d'une alerte d'état quand sa règle n'est plus vérifiée, ou quand sa mission n'est plus en cours.
- **F14. Pas de faux « résolu »** : une alerte d'état n'est close que sur un instantané complet. Une alerte d'événement (incident) n'est jamais close automatiquement tant que la mission est active.

5.4 Notifications

- **F15.** Une nouvelle alerte déclenche les notifications du bloc `notify` de sa règle : rôle (ou utilisateur), niveau de management, canal (`in_app` , `sms` , `email` , `call`).
- **F16.** Les destinataires sont résolus dans le TMS au moment de l'envoi. L'adresse dépend du canal : téléphone pour SMS et appel, e-mail pour e-mail, identifiant pour in-app.
- **F17.** Chaque notification est enregistrée avec son statut (`delivered` ou `failed`). Un échec ne recrée pas l'alerte : il est journalisé pour reprise humaine.
- **F18.** Une alerte supprimée par déduplication ne renotify personne.

5.5 Enquêtes du LLM

- **F19.** Chaque nouvelle alerte de sévérité $\geq$ `investigate_min_severity` (`medium` par défaut) est confiée à l'enquêteur, au plus 4 par cycle, les plus graves d'abord.
- **F20.** Les enquêtes tournent en arrière-plan (2 en parallèle, 8 en attente au plus) : la détection du cycle suivant n'attend jamais le LLM.
- **F21.** L'enquêteur reçoit l'alerte, ses notifications et des faits clés calculés de façon déterministe. Il peut lire le TMS, l'historique de l'agent, évaluer les règles sur la mission et appeler le chauffeur sous garde-fous.
- **F22.** L'enquête se termine par une évaluation structurée (`submit_assessment`) jointe à l'alerte : synthèse, situation, cause probable, niveau de risque, impact client, actions recommandées, preuves, appel chauffeur, suspicion de fausse alerte, confiance.
- **F23.** Le LLM ne peut ni créer d'alerte ni notifier : `create_alert` et `notify_recipients` lui sont interdits.

5.6 Assistant opérateur

- **F24.** L'assistant répond en langage naturel (`ask` pour une question, `chat` pour une conversation, `report` pour un rapport de situation).
- **F25.** Chaque conversation garde son historique en base, y compris après un redémarrage.
- **F26.** Sur demande explicite, l'assistant peut clore une alerte, appeler un chauffeur ou journaliser une décision. Il n'émet jamais d'alerte ni de notification.

5.7 Traçabilité

- **F27.** Toute action de l'agent est journalisée (alertes, notifications, clôtures, enquêtes, appels, échecs) avec la mission et l'alerte concernées.

6. Moteur de règles déclaratif

Le moteur (`tms_agent.rules_engine`) est déterministe, sans LLM ni dépendance MCP. Il reçoit un contexte (mission, véhicule, écart, ETA, chauffeur, événement) et l'heure TMS, et renvoie les règles déclenchées avec leurs données observées, leur message rendu et leurs destinataires.

6.1 Format d'une règle

Une règle est un fichier YAML dans `agent/domain/rules/`, validé strictement au chargement (Pydantic). Exemple :

```
id: deviation_persistent
version: 2
enabled: true
severity: high # low | medium | high | critical
category: route_deviation
author: équipe exploitation
description: >
  Alerte si le véhicule s'écarte de plus de 5 km de l'itinéraire pendant plus
  de 20 minutes.
when:
  all: # all (ET) | any (OU) | not, imbricables
    - { field: mission.status, operator: equals, value: in_progress }
    - { field: deviation.current_offset_km, operator: greater_than, value: 5
  }
  - { field: deviation.duration_minutes, operator: greater_than, value: 20
```

```

}
observe:                                # champs supplémentaires copiés dans l'alerte
- deviation.vehicle_position
then:
  action: emit_alert                    # emit_alert | log_only | create_report
  alert:
    title: "Dérive d'itinéraire"
    message: >
      Le véhicule {{ vehicle.plate }} de la mission {{ mission.reference }}
      s'est écarté de {{ deviation.current_offset_km }} km.
    deduplication_key: "deviation:{{ mission.id }}"
    cooldown_minutes: 30
  notify:
    - { role: driver, management_level: M+0, channel: call }
    - { role: ops_agent, management_level: M+1, channel: in_app }
    - { role: ops_supervisor, management_level: M+2, channel: sms }

```

Les gabarits `{{ champ }}` sont de simples substitutions de chemins, sans code exécutable. Une règle dont une condition porte sur `event.*` est une **règle d'événement** ; les autres sont des **règles d'état**.

6.2 Opérateurs

Famille	Opérateurs
Comparaison	<code>equals</code> , <code>not_equals</code> , <code>greater_than</code> , <code>greater_or_equal</code> , <code>less_than</code> , <code>less_or_equal</code> , <code>between</code>
Appartenance	<code>in</code> , <code>not_in</code>
Temporel	<code>older_than_minutes</code> , <code>newer_than_minutes</code> , <code>duration_greater_than_minutes</code>
Géospatial	<code>distance_greater_than_km</code> (haversine), <code>inside_polygon</code> , <code>outside_polygon</code>
Logique	<code>all</code> , <code>any</code> , <code>not</code>

Chaque opérateur valide le type de sa valeur au chargement. Un champ absent du contexte rend la condition fausse, sans erreur.

6.3 Champs dérivés

Le moteur calcule des champs que le TMS ne fournit pas directement, à partir de l'heure TMS :

Champ	Calcul
<code>mission.last_event_type</code> , <code>mission.last_event_age_minutes</code>	Dernier événement de la mission et son âge
<code>vehicle.last_seen_age_minutes</code>	Âge de la dernière position reçue
<code>deviation.duration_minutes</code>	Fourni, ou calculé depuis <code>current_offset_since</code>
<code>eta.delay_minutes</code>	Fourni, ou <code>current_eta</code> - <code>planned_delivery_at</code>
<code>event.age_minutes</code> , <code>event.distance_to_target_km</code>	Âge de l'événement, distance au point de chargement ou de livraison visé

6.4 Règles du POC

Règle	Type	Sévérité	Condition	Cooldown	Notifiés
<code>critical_incident</code>	Événement	critical	message chauffeur de sévérité <code>critical</code>	0 (clé par événement)	M+1 in-app, M+2 SMS, M+3 appel, M+4 e-mail
<code>deviation_persistent</code>	État	high	écart > 5 km depuis > 20 min	30 min	M+0 appel, M+1 in-app, M+2 SMS
<code>immobilization_anomaly</code>	État	high	véhicule <code>stopped</code> , aucun événement depuis > 120 min	60 min	M+0 appel, M+1 in-app
<code>eta_delay</code>	État	medium	retard ETA > 45 min	60 min	M+1 in-app, M+3 e-mail
<code>data_gap_moving</code>	État	medium	véhicule <code>moving</code> , aucun événement depuis > 180 min	120 min	M+1 in-app
<code>arrival_out_of_zone</code>	Événement	medium	arrivée déclarée à > 2 km du point visé	0 (clé par événement)	M+1 in-app, M+2 SMS

Ces seuils sont des hypothèses de démonstration. Ils seront remplacés par les vraies règles après les entretiens avec les agents de suivi.

6.5 Déduplication et cooldown

- `deduplication_key` : si une alerte ouverte porte déjà cette clé, la nouvelle n'est pas émise.
- `cooldown_minutes` : délai minimal, en temps TMS, avant de réémettre une alerte de même clé, même close.
- La vérification et la création sont atomiques côté serveur MCP (verrou consultatif PostgreSQL par clé) : deux instances concurrentes ne créent pas de doublon.
- Un recul de l'horloge TMS (remise à zéro de la simulation) est détecté et ne bloque pas les alertes.

6.6 Scénarios métier

Chaque règle est testée sur des données figées dans `agent/domain/scenarios/`. Un fichier décrit un instantané de base, puis des cas qui le modifient et listent les règles attendues. `rules-engine test domain/rules domain/scenarios` exécute les 20 cas actuels et sort en erreur si un cas échoue.

6.7 Cycle de vie d'une règle

1. **Rédaction** par l'équipe métier, ou avec elle.
2. **Test** : `rules-engine validate`, puis rejeu des scénarios ; `rules-engine eval --explain` détaille chaque condition sur un instantané.
3. **Revue** humaine de la règle et de ses seuils (revue Git).
4. **Déploiement** : dépôt du fichier, rechargement à chaud au cycle suivant.
5. **Observation** du taux de faux positifs (le champ `false_positive_suspected` des enquêtes y aide).
6. **Ajustement** : nouvelle `version`. Chaque alerte garde la version qui l'a déclenchée.

7. Serveur MCP

Le serveur MCP (`mcp_server/`, paquet `tms_mcp`) est le seul point d'accès de l'agent au TMS et à la base de l'agent. Il expose 25 tools sémantiques, en stdio (développement) ou en Streamable HTTP sur le port 8002 (déploiement).

7.1 Contrat connecteur pivot

Le serveur ne parle au TMS qu'à travers l'interface `TmsConnector`, en lecture seule. Elle renvoie des dictionnaires au format pivot (mission, événement, véhicule, position, itinéraire, écart, ETA, chauffeur, utilisateur) :

- `health`, `now` : santé et heure courante du TMS.
- `list_missions`, `get_mission`, `list_mission_events`, `get_mission_route`, `get_mission_deviation`, `get_mission_eta`.
- `list_vehicles`, `get_vehicle`, `get_vehicle_position`, `list_vehicle_positions`.
- `get_driver`, `get_user`, `list_users`, `list_role_users`.

L'implémentation REST (`RestConnector`, httpx) gère la pagination par curseur, les erreurs RFC 7807, le timeout (10 s) et 3 réessais sur erreur réseau, 429 et 5xx. **Brancher un nouveau TMS = écrire un connecteur** qui traduit ses ressources vers ce contrat. Les tools, l'agent et les règles ne changent pas.

7.2 Catalogue des tools

Famille A : TMS (lecture seule)

Tool	Usage
<code>get_tms_time</code>	Heure courante du TMS (référence de toute la logique temporelle)
<code>get_active_missions</code>	Missions en cours, point d'entrée de chaque cycle
<code>get_mission_snapshot</code>	Instantané complet d'une mission en un appel (mission, événements, véhicule, écart, ETA, chauffeur, erreurs partielles)
<code>list_missions</code> , <code>get_mission_details</code>	Missions tous statuts, détail d'une mission
<code>get_mission_events</code>	Chronologie des événements (filtre <code>since</code>)
<code>get_mission_route</code> , <code>get_mission_deviation</code> , <code>get_mission_eta</code>	Itinéraire planifié, écart à l'itinéraire, ETA recalculée

Tool	Usage
<code>get_vehicle_details</code> , <code>get_vehicle_position</code> , <code>get_vehicle_positions</code>	Véhicule, dernière position, historique GPS
<code>get_driver_details</code> , <code>list_users</code>	Fiche chauffeur, utilisateurs par rôle ou niveau

Famille B : orchestrateur (base de l'agent)

Tool	Usage	Autorisé au LLM
<code>create_alert</code>	Créer une alerte avec déduplication et cooldown atomiques	Non (boucle seule)
<code>notify_recipients</code>	Résoudre les destinataires, envoyer, enregistrer	Non (boucle seule)
<code>list_alerts</code> , <code>get_alert</code> , <code>list_alert_notifications</code>	Consulter les alertes et leurs notifications	Oui
<code>resolve_alert</code>	Clore une alerte avec un motif	Oui (assistant, sur demande)
<code>annotate_alert</code>	Joindre l'analyse d'enquête	Boucle
<code>call_driver</code>	Appel chauffeur simulé, sous garde-fous	Oui
<code>log_agent_action</code> , <code>list_agent_actions</code>	Écrire et consulter le journal d'audit	Oui
<code>get_operations_overview</code>	Tableau de bord : alertes ouvertes par sévérité, indicateurs interprétés de chaque mission en cours (retard, écart, véhicule)	Oui

7.3 Description sémantique des tools

L'agent découvre les tools à l'exécution (`list_tools`). Chaque tool porte donc tout ce qu'il faut pour être bien utilisé sans code spécifique :

- un nom verbe + objet (`get_mission_eta`) et un titre court ;
- une description qui dit l'intention métier et le moment d'usage, pas seulement ce que fait l'API ;
- un `inputSchema` documenté champ par champ et un `outputSchema` ;
- les annotations MCP (`readOnlyHint` , `destructiveHint` , `idempotentHint`) ;
- des métadonnées `_meta` : `domain` , `category` , `backend` , `read_only` , `latency_hint` , `call_frequency_hint` , `cost_hint` .

7.4 Garde-fous côté serveur

- **Lecture seule sur le TMS** : aucun tool n'écrit dans le TMS.
- `call_driver` refuse l'appel sans alerte ouverte de sévérité `high` ou `critical` sur la mission, et ne rappelle pas le même chauffeur pour la même mission à moins de 15 minutes d'intervalle.
- `create_alert` sérialise la déduplication par clé (verrou consultatif PostgreSQL).
- Les erreurs sont renvoyées comme résultats d'erreur MCP lisibles par le LLM, jamais comme crash du serveur.

7.5 Notifications

`NotificationService` résout les destinataires en lisant le TMS, choisit l'adresse selon le canal, envoie via un `ChannelSender` et enregistre chaque notification. Le POC utilise `SimulatedSender` (aucun envoi réel, délai de livraison réaliste par canal). Brancher un fournisseur réel (SMTP, SMS, téléphonie, Teams) revient à fournir un autre `ChannelSender` .

8. Agent IA

L'agent (`agent/` , paquet `tms_agent`) combine une boucle de surveillance déterministe et deux usages du LLM, l'enquêteur et l'assistant. Les trois sont des graphes LangGraph et ne voient le TMS qu'à travers le serveur MCP.

8.1 Boucle de surveillance

Un cycle = une invocation du graphe `observe → evaluate → act → analyze → report` :

Nœud	Rôle	Sortie
<code>observe</code>	<code>get_active_missions</code> , puis <code>get_mission_snapshot</code> par mission (concurrency bornée)	Instantanés, erreurs partielles, <code>tms_time</code>
<code>evaluate</code>	Rechargement à chaud des règles, évaluation des règles d'état et d'événement sur les nouveaux événements	Déclenchements triés par sévérité
<code>act</code>	<code>create_alert</code> → <code>notify_recipients</code> , journalisation des <code>log_only</code> , événements marqués traités, clôture automatique	Alertes émises, supprimées, closes
<code>analyze</code>	Lance les enquêtes sur les nouvelles alertes (arrière-plan par défaut)	Enquêtes en cours
<code>report</code>	Bilan du cycle, état persistant	Événements de suivi (console, logs)

Si `observe` échoue (TMS ou MCP injoignable), le graphe saute directement à `report`. `analyze` n'est traversé que si de nouvelles alertes le justifient et que le LLM est disponible ; sinon l'agent revient vers le LLM tous les `llm_reprobe_every_cycles` cycles (6).

8.2 Enquêteur

Sous-graphe ReAct (modèle ⇌ tools) qui analyse une alerte et se termine obligatoirement par `submit_assessment` :

1. Il reçoit l'alerte, ses notifications et les **faits clés** calculés par `mission_facts` (retard ou avance, écart, âge du dernier événement, vitesse). Un petit modèle n'a ainsi rien à recalculer.
2. Il lit le TMS et l'historique de l'agent, et explique les règles sur la mission (`explain_rules_for_mission`).
3. Il peut appeler le chauffeur si la situation le justifie (garde-fous du serveur).
4. Il remet son évaluation : schéma strict en profil complet, validation tolérante en profil compact (synonymes français du risque, listes écrites en texte, confiance en pourcentage).

Garde-fous du graphe ReAct :

- au plus `llm_max_turns` appels au modèle (10) ;
- tools exécutés en parallèle ; une erreur de tool est renvoyée au modèle, jamais levée ;
- arguments JSON illisibles renvoyés au modèle pour correction ;
- un modèle qui conclut en texte est relancé, puis l'appel à `submit_assessment` est forcé (décodage guidé chez vLLM) ;
- au dernier tour autorisé, l'évaluation est forcée pour conclure avec ce qui a été collecté ;
- résultats de tools tronqués au-delà de 24 000 caractères (5 000 en profil compact).

8.3 Assistant opérateur

Même graphe ReAct, sans tool terminal, avec un checkpointer PostgreSQL : chaque fil (`thread_id`) garde son historique. Commandes : `ask` (une question), `chat` (conversation, `/nouveau` , `/quitter`), `report` (rapport de situation). Le prompt système lui interdit d'inventer des données et lui impose de citer les identifiants et les chiffres lus.

8.4 Fournisseurs LLM

Fournisseur	Client	Quand	Particularités
<code>local</code> (défaut)	SDK <code>openai</code> , toute API compatible (vLLM, Ollama)	Toujours autorisé : les données restent internes	Fenêtre lue sur <code>/v1/models</code> , budget de contexte calibré sur les tokens réels, raisonnement Qwen3 selon l'effort, appel de tool forcé
<code>anthropic</code>	SDK <code>anthropic</code> (Claude)	Seulement si la politique de données l'autorise	Effort par usage (enquête <code>high</code> , chat <code>medium</code>), 16 000 tokens de sortie

Les deux implémentent la même interface (`LlmClient` , format interne `tool_use` / `tool_result`). Si la conversation dépasse la fenêtre, les plus anciens résultats de tools sont remplacés par une mention « retiré » dans la requête, sans modifier l'historique.

8.5 Profils d'outils

La boîte à outils (`tms_agent.tools`) assemble les tools MCP découverts et les tools locaux (`list_business_rules` , `explain_rules_for_mission` , `submit_assessment`), puis les filtre par mode :

- **Interdits au LLM, dans tous les modes** : `create_alert` , `notify_recipients` .
- **Profil complet** : tous les tools autorisés, descriptions enrichies du titre et des métadonnées MCP.
- **Profil compact** (automatique sous 32k tokens de fenêtre) : 8 tools pour l'enquête, 11 pour l'assistant, descriptions réduites à leur première phrase, schémas sans titres, résultats JSON épurés des valeurs vides.

8.6 Interface en ligne de commande

Commande	Usage
<code>tms-agent watch [--interval N] [-v]</code>	Surveillance continue
<code>tms-agent cycle</code>	Un seul cycle, bilan JSON
<code>tms-agent ask "...", chat, report</code>	Assistant opérateur
<code>tms-agent rules, tools</code>	Règles actives, tools vus par le LLM
<code>tms-agent reset-state -y</code>	Oublie les événements déjà traités
<code>--no-llm, --provider, --model</code>	Mode déterministe, choix du fournisseur et du modèle
<code>rules-engine validate eval test</code>	Outils de l'équipe métier sur les règles

9. Données et persistance

Toutes les données produites par l'orchestrateur vivent dans une base PostgreSQL 16 (`tms_agent_db`), avec un schéma par responsabilité. Le TMS reste la source de vérité des missions : l'orchestrateur n'en garde aucune copie, il relit le TMS à chaque cycle.

Schéma	Propriétaire	Tables	Contenu
agent	Serveur MCP	alerts , notifications , agent_logs , driver_calls	Alertes et analyses, notifications envoyées, journal d'audit, appels chauffeur
monitor	Boucle de surveillance	processed_events , kv	Événements déjà évalués par mission, curseurs de la boucle
assistant	Assistant	Tables du checkpointer LangGraph	Historique des conversations par thread_id

Les schémas et tables sont créés au démarrage de façon idempotente. Chaque composant n'accède qu'à son schéma ; l'agent ne lit les alertes qu'à travers les tools MCP.

9.1 Alerte (agent.alerts)

Champ	Type	Rôle
id	texte (alr_...)	Identifiant
rule_id , rule_version	texte, entier	Règle et version qui l'ont déclenchée (audit)
mission_id , event_id	texte	Mission du TMS ; événement déclencheur pour une règle d'événement
severity , category	texte	low à critical ; regroupement métier
title , message	texte	Rendus depuis les gabarits de la règle
observed_data	JSONB	Valeurs lues dans le TMS qui ont déclenché la règle
deduplication_key	texte	Clé de déduplication (index avec created_at)

Champ	Type	Rôle
<code>status</code>	texte	<code>open</code> ou <code>resolved</code>
<code>created_at</code> , <code>resolved_at</code> , <code>resolution_reason</code>	horodatages TMS, texte	Cycle de vie et motif de clôture
<code>analysis</code>	JSONB	Évaluation de l'enquêteur, avec sa durée

9.2 Autres tables

- **`notifications`** : alerte, destinataire (identifiant, nom, rôle, niveau), canal, adresse, message, statut (`delivered` ou `failed`), erreur, envoi et livraison.
- **`agent_logs`** : journal d'audit append-only (type, mission, alerte, charge JSONB, date). Types : `rule_observation`, `report`, `notification_failed`, `investigation_failed`, décisions de l'assistant, etc.
- **`driver_calls`** : mission, alerte, chauffeur, téléphone, motif, statut, début et fin d'appel.
- **`monitor.processed_events`** : clé (`mission_id`, `event_id`). Garantit qu'une règle d'événement ne se déclenche qu'une fois par événement, y compris après redémarrage.

9.3 Conventions

- Dates en `TIMESTAMPZ`, écrites en heure TMS (UTC, ISO 8601).
- Données semi-structurées (données observées, analyses, charges du journal) en `JSONB`.
- Identifiants préfixés par type (`alr_`, `ntf_`, ...) pour être lisibles dans les logs et les réponses du LLM.
- Aucune donnée n'est écrite dans le TMS.

10. Stack technique

Tout l'orchestrateur est en Python, dans un seul espace de travail uv : un environnement et un `uv.lock` communs, deux paquets déployés séparément. Versions verrouillées au 6 octobre 2026.

Couche	Technologie	Version	Usage
Langage	Python	≥ 3.11 (3.13 en local)	Agent et serveur MCP
Outils	uv (espace de travail), hatchling	0.12	Environnement unique, verrouillage, build des deux paquets
Orchestration agentique	LangGraph	1.2.12	Boucle de surveillance, graphes ReAct de l'enquêteur et de l'assistant
Mémoire de conversation	langgraph-checkpoint-postgres	3.1.2	Historique de l'assistant (schéma <code>assistant</code>)
Protocole d'outils	MCP, SDK Python <code>mcp</code>	2.3.0	Tools sémantiques, transports stdio et Streamable HTTP
Serveur HTTP MCP	uvicorn, Starlette	0.54 / 1.7	Transport HTTP (port 8002)
Connecteur TMS	httpx	0.28.1	Client REST : pagination, réessais, timeouts
LLM local	SDK <code>openai</code> + vLLM	3.24.0 / 0.30	API compatible OpenAI, parsers de tools <code>qwen3_coder</code> et de raisonnement <code>qwen3</code>
Modèle de développement	Qwen3.5-2B (FP8)	—	Fenêtre de 8k tokens, sur GPU NVIDIA via Docker
LLM cloud (option)	SDK <code>anthropic</code> , Claude	1.11.0	Seulement si la politique de données l'autorise
Validation	Pydantic, pydantic-settings	2.13.5 / 2.15.0	Règles, arguments de tools, évaluations, configuration
Règles	PyYAML	6.0.3	Lecture des règles et des scénarios

Couche	Technologie	Version	Usage
Base de données	PostgreSQL, psycopg 3, psycopg-pool	16 / 3.3.6 / 3.3.3	Base de l'agent, pools de connexions, JSONB, verrous consultatifs
Tests	pytest, pytest-asyncio	9.1.1 / 1.4.0	101 tests (82 agent, 19 serveur MCP)
TMS simulé (hors dépôt)	FastAPI, SQLAlchemy 2, SQLite, Typer ; React 19, Vite, MapLibre	—	API de test, simulateur de missions, visualisation

Ports locaux : TMS 8000, vLLM 8001, serveur MCP 8002, agent 8003 (réservé à la future API), interface TMS 5173.

Choix structurants

- **LangGraph** plutôt qu'une boucle maison : états typés, branchements explicites, checkpointer PostgreSQL prêt à l'emploi.
- **MCP** entre l'agent et le TMS : découverte dynamique des tools, transport interchangeable, serveur déployable à part.
- **LLM local par défaut** : confidentialité des données d'exploitation et des chauffeurs ; le cloud reste une option explicite.
- **Règles en YAML validé par Pydantic** : lisibles par le métier, erreurs de chargement explicites, aucun code exécutable.
- **PostgreSQL unique** : un seul système à exploiter pour l'état, l'audit et la mémoire, séparés par schéma.

11. Exigences non fonctionnelles

La détection doit rester rapide et fiable quel que soit l'état du LLM : c'est elle qui porte le service, l'analyse vient en plus. Les cibles ci-dessous sont à valider en pilote sur le volume réel du TMS cible.

11.1 Performance

Exigence	Cible	Moyen
Délai entre l'anomalie visible dans le TMS et l'alerte	≤ 1 cycle (10 s par défaut)	Cycle court, règles en mémoire, notification dans le même cycle
Durée d'un cycle	< intervalle de polling	Instantané en un appel par mission, lectures parallèles bornées (<code>mission_concurrency</code>)
La détection n'attend jamais le LLM	Toujours	Enquêtes en arrière-plan, file bornée (8), au plus 4 nouvelles par cycle
Analyse jointe à une nouvelle alerte	< 2 min avec le modèle local	2 enquêtes parallèles, 10 appels au modèle au plus, profil compact
Contexte LLM maîtrisé	Aucun dépassement de fenêtre	Budget calibré sur les tokens réels, troncature des résultats, faits précalculés

Leviers de montée en charge, dans l'ordre : augmenter `mission_concurrency` ; filtrer les missions sur `updated_since` ; répartir les missions entre instances (§ 14).

11.2 Fiabilité

- **Isolation des pannes** : une mission illisible, un tool en erreur ou une enquête ratée n'arrête pas le cycle.
- **Reprise** : TMS ou MCP injoignable → backoff exponentiel et reconnexion MCP automatique. LLM indisponible → mode déterministe, nouvel essai tous les 6 cycles.
- **Au moins une fois, sans doublon** : un événement n'est marqué traité qu'après l'enregistrement de ses alertes ; la déduplication atomique empêche les doublons en cas de reprise.
- **Pas de faux « résolu »** : clôture automatique seulement sur instantané complet.
- **Robustesse des règles** : une règle invalide ne remplace jamais le jeu actif.
- **Arrêt propre** : les enquêtes en cours ont un délai de grâce avant annulation.

11.3 Sécurité et confidentialité

- **Moindre privilège sur le TMS** : lecture seule, par construction du connecteur.
- **Le LLM ne décide pas** : `create_alert` et `notify_recipients` lui sont inaccessibles ; `call_driver` est borné par le serveur.
- **Injection de prompt** : les résultats de tools (messages chauffeur, champs libres) sont des données, jamais des instructions. Le prompt système l'impose et aucune action sensible ne dépend du seul texte du LLM.
- **Données personnelles** (chauffeurs, téléphones, positions) : LLM local par défaut, cloud seulement sur décision explicite. Durée de conservation à définir (RGPD).
- **Secrets** : fichiers `.env` ignorés par git en POC ; gestionnaire de secrets en production.
- **Transport MCP HTTP** : à placer derrière TLS avec authentification (jeton ou OAuth) dès le pilote.

11.4 Auditabilité et observabilité

- Chaque alerte porte sa règle, sa version, ses données observées, ses notifications et son analyse : on sait toujours pourquoi l'agent a alerté et qui a été prévenu.
- Le journal d'audit trace toutes les actions de l'agent, y compris ses échecs.
- POC : logs texte et bilan de cycle sur la console. Production : métriques (missions surveillées, alertes émises, durée de cycle, disponibilité du LLM), traces de cycle et d'appels de tools, suivi des appels LLM (latence, tokens).

11.5 Maintenabilité et portabilité

- Aucun code spécifique au TMS dans l'agent : un nouveau TMS = un connecteur.
- Un nouveau canal de notification = un `ChannelSender`. Un nouveau fournisseur LLM = un `LlmClient`.
- Le moteur de règles ne dépend ni du LLM ni de MCP : il se teste seul.
- Configuration entièrement par variables d'environnement (préfixe `AGENT_` pour l'agent, `POSTGRES_*` partagées).

12. Déploiement, configuration et exploitation

L'agent et le serveur MCP se déploient dans deux conteneurs séparés, construits depuis le même `uv.lock` : les versions sont identiques partout. En développement, l'agent lance le serveur MCP en sous-processus (stdio) depuis le même environnement.

12.1 Topologie cible

Service	Image / installation	Port	Dépend de
mcp-server	uv sync --frozen --no-dev --package tms-mcp-server	8002 (HTTP)	API du TMS, PostgreSQL
agent	uv sync --frozen --no-dev --package tms-agent	8003 (future API)	mcp-server , PostgreSQL, vLLM
postgres	PostgreSQL 16	5432	—
vllm	vllm/vllm-openai , GPU NVIDIA	8001	—

En déploiement, l'agent joint le serveur MCP en HTTP (`AGENT_MCP_TRANSPORT=http`). Une seule instance de la boucle de surveillance tourne à la fois (§ 14).

12.2 Configuration

Variable	Défaut	Service	Rôle
<code>TMS_API_URL</code>	<code>http://127.0.0.1:8000/v1</code>	MCP	API du TMS
<code>TMS_TIMEOUT_S</code> , <code>TMS_MAX_RETRIES</code>	10, 3	MCP	Timeout et réessais vers le TMS
<code>DRIVER_CALL_MIN_INTERVAL_MIN</code>	15	MCP	Intervalle minimal entre deux appels au même chauffeur
<code>MCP_HOST</code> , <code>MCP_PORT</code>	127.0.0.1, 8002	MCP	Écoute HTTP
<code>POSTGRES_DB</code> , <code>_USER</code> , <code>_PASSWORD</code> , <code>_HOST</code> , <code>_PORT</code>	<code>tms_agent_db</code> , <code>postgres</code> , <code>—</code> , <code>localhost</code> , <code>5432</code>	Les deux	Base de l'agent
<code>AGENT_MCP_TRANSPORT</code> , <code>AGENT_MCP_URL</code>	<code>stdio</code> , <code>http://127.0.0.1:8002/mcp</code>	Agent	Accès au serveur MCP
<code>AGENT_RULES_DIR</code>	<code>agent/domain/rules</code>	Agent	Dossier des règles
<code>AGENT_POLL_INTERVAL_S</code> , <code>AGENT_MISSION_CONCURRENCY</code>	10, 6	Agent	Rythme et parallélisme de la boucle
<code>AGENT_LLM_PROVIDER</code> , <code>AGENT_LLM_ENABLED</code>	<code>local</code> , <code>true</code>	Agent	Fournisseur LLM, mode déterministe
<code>AGENT_LOCAL_BASE_URL</code> , <code>AGENT_LOCAL_MODEL</code>	<code>http://localhost:8001/v1</code> , <code>Qwen/Qwen3.5-2B</code>	Agent	LLM local
<code>AGENT_TOOLSET</code>	<code>auto</code>	Agent	Profil d'outils (<code>auto</code> , <code>full</code> , <code>compact</code>)
<code>AGENT_INVESTIGATE_MIN_SEVERITY</code>	<code>medium</code>	Agent	Seuil d'enquête automatique
<code>ANTHROPIC_API_KEY</code>	<code>—</code>	Agent	Seulement avec <code>AGENT_LLM_PROVIDER=anthropic</code>

12.3 Exploitation courante

- **Modifier une règle** : déposer le fichier YAML ; rechargement au cycle suivant, sans redémarrage.
- **Changer de modèle** : `AGENT_LOCAL_MODEL` (ou `--model`), le profil d'outils s'adapte à la fenêtre lue sur vLLM.
- **Remettre à zéro** : `tms-mcp reset-db -y` (alertes, notifications, journal) et `tms-agent reset-state -y` (événements traités).
- **Consulter** : `tms-mcp alerts [--status open]`, `tms-agent report`.
- **Démonstration** : TMS simulé à ×30 avec les 4 scénarios, puis `tms-agent watch --interval 5`. En quelques minutes : incident critique notifié à 4 niveaux, dérive puis clôture, retard ETA, immobilisation, chacun suivi de son analyse.

13. Stratégie de tests et qualité

Le système se teste sans TMS réel ni LLM réel : un faux TMS en mémoire, un serveur MCP en mémoire et un LLM scripté couvrent la boucle complète. Aujourd’hui : 101 tests automatisés (82 agent, 19 serveur MCP) et 20 scénarios métier, tous au vert.

Niveau	Ce qui est vérifié	Doublures
Moteur de règles	Opérateurs, chargement et erreurs lisibles, évaluation, déduplication, scénarios métier	Aucune (fonctions pures)
Scénarios métier	20 cas sur données figées, maintenus avec l'équipe métier	Aucune
Serveur MCP	Connecteur REST (pagination, réessais), tools, garde-fous, base de l'agent	Transport HTTP simulé, PostgreSQL réel
Boucle de surveillance	De bout en bout : mission nominale silencieuse, alertes, déduplication, clôture, rechargement à chaud, pannes	Faux TMS → serveur MCP en mémoire → graphe LangGraph
Chemins LLM	Enquête, assistant, relance et appel forcé, arguments invalides, profil compact, budget de contexte, validation tolérante	LLM scripté, réponses vLLM simulées

Chaque test PostgreSQL travaille dans un schéma jetable, supprimé à la fin.

Commandes

```
uv sync                                     # à la racine :
environnement unique
(cd agent && uv run pytest)                  # 82 tests
(cd mcp_server && uv run pytest)              # 19 tests
(cd agent && uv run rules-engine test domain/rules domain/scenarios)
```

À ajouter avant le pilote

- ☐ Intégration continue : lint (ruff), typage (mypy), tests avec PostgreSQL en service, scénarios métier.

- ☐ Jeu d'évaluation d'enquêtes annotées, pour mesurer la qualité des analyses et comparer les modèles.
- ☐ Tests de contrat du connecteur sur le TMS cible (format pivot, pagination, erreurs).
- ☐ Test de charge : durée de cycle selon le nombre de missions actives.

14. Risques, limites et feuille de route

Le POC valide l'architecture, pas l'exploitation en production. Le principal risque est métier : les vraies règles et leurs seuils restent à formaliser avec les agents de suivi.

14.1 Risques

Risque	Impact	Parade
Règles du POC mal calibrées pour le terrain	Faux positifs, fatigue d'alerte, perte de confiance	Entretiens avec les agents de suivi, scénarios rejoués, suivi de <code>false_positive_suspected</code> , versions de règles
TMS cible éloigné du contrat pivot (données manquantes : écart, ETA)	Connecteur plus coûteux, règles inapplicables	Audit de l'API cible tôt ; calcul de l'écart ou de l'ETA dans le connecteur si besoin
Qualité d'enquête d'un modèle de 2B paramètres et 8k de contexte	Analyses superficielles	Modèle plus grand (8B à 32B, 32k), jeu d'évaluation ; la détection n'en dépend pas
Volume de missions et limites de débit du TMS	Cycles trop longs, 429	Concurrence bornée, <code>updated_since</code> , cache des données de référence, répartition entre instances
Une seule instance de la boucle	Pas de haute disponibilité	Verrou par mission (<code>pg_try_advisory_lock</code>) ou file de travail
Données personnelles des chauffeurs	Non-conformité RGPD	LLM local, durée de conservation, purge, journal d'audit consultable
Notifications simulées	Rien ne part réellement	<code>ChannelSender</code> réels au pilote, mode bac à sable conservé

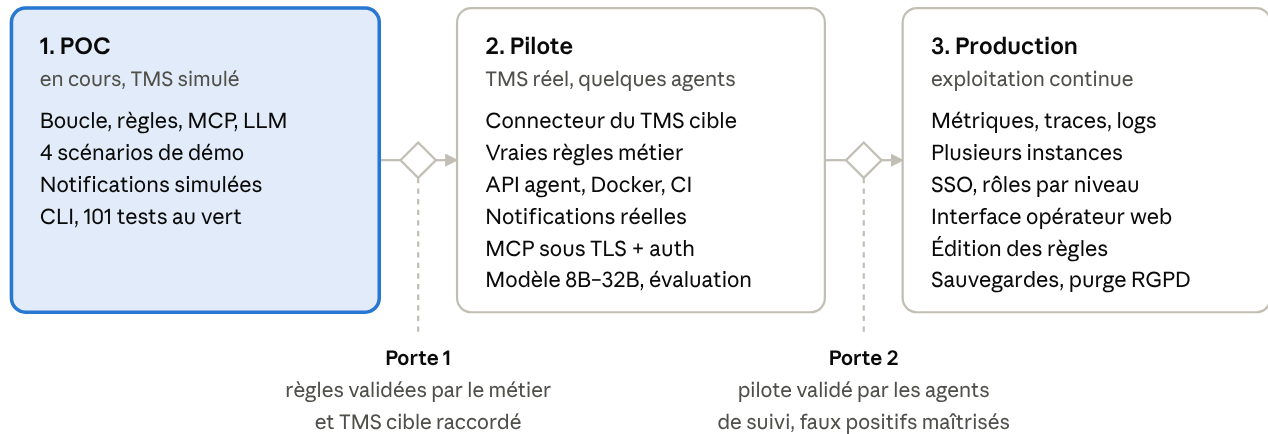
14.2 Limites actuelles

- Pas d'API HTTP pour l'agent ni d'interface opérateur : usage en ligne de commande.
- Transport MCP HTTP sans TLS ni authentification.
- Pas de migrations versionnées (tables créées au démarrage), ni de sauvegarde ou de purge.
- Le serveur MCP accède à la base en synchrone depuis du code asynchrone.
- Logs texte uniquement : pas de métriques, de traces ni de suivi des appels LLM.
- Pas d'utilisateurs ni de droits côté opérateur ; journal d'audit non exposé.

14.3 Feuille de route

Trois phases, chacune ouverte par une porte de passage. Ce qui ne change pas d'une phase à l'autre : accès au TMS uniquement par MCP, règles déterministes pour décider et LLM pour expliquer, LLM local par défaut.

Trois phases : POC, pilote sur un TMS réel, production



feuille de route · 3 phases, 2 portes

Le pilote ne démarre qu'avec des règles validées par le métier et un connecteur du TMS cible. La production suppose un pilote jugé utile par les agents de suivi.